

PREMIER UNIVERSITY CHITTAGONG

Department of Computer Science & Engineering



Project Report

Course Name : Big Data Analytics Laboratory
Course Code : CSE 4346
Name of Report : Big Data Analysis of Spotify Tracks Using
Hadoop MapReduce and Apache Pig.
Date of Submission : 15.09.2026

Submitted By,	
Name	: Md. Samin Osman
ID	: 0222220005101042
Semester	: 8 th
Batch	: 42 nd
Session	: Spring - 26
Section : A	

<u>Submitted To :</u> MD Tamim Hossain Lecturer , Department of Computer Science & Engineering	Remarks
--	----------------

Problem Statement

The objective of this laboratory exercise is to perform Big Data processing and analysis using Apache Hadoop, MapReduce, and Apache Pig on a large Spotify tracks dataset. The dataset contains information such as track ID, track name, artist name, album name, popularity, duration, explicit content, danceability, energy, acousticness, genre, and other audio features. The tasks involve developing and executing distributed data-processing programs to extract meaningful information and insights from the dataset.

Specifically, the laboratory focuses on:

- Finding the most popular song based on the popularity value using Hadoop MapReduce.
- Performing popularity summarization for different music genres by calculating statistical information related to track popularity.
- Identifying the top artists based on the number of tracks associated with each artist.
- Using Apache Pig to identify the top songs based on popularity.
- Finding the top artists based on the number of tracks in the dataset.
- Performing genre-based statistical analysis to analyze music tracks across different genres.
- Identifying the top music genres based on the number of tracks available in the dataset.
- Analyzing explicit and non-explicit tracks to understand their distribution within the Spotify dataset.
- Using Apache Pig to perform filtering, grouping, sorting, counting, and aggregation operations on the dataset.
- Executing the developed MapReduce and Pig programs through Hadoop/HDFS and analyzing their outputs and execution results.

The overall goal of the project is to demonstrate how distributed Big Data technologies can efficiently process and analyze a large music dataset to discover meaningful patterns related to song popularity, artists, genres, and explicit content.

Dataset Description

The dataset used for this project is the Spotify Tracks Dataset, obtained from Kaggle (<https://www.kaggle.com/datasets/saichaitanyareddyai/spotify-tracks-dataset-audio-features>). It contains 114,000 track records (104,253 non-explicit and 9,747 explicit tracks) spread across 114 music genres, with each record describing a track along with its artist, album, popularity score, and a set of Spotify audio features. The field-level structure of the dataset is described below.

Field	Description
track_id	Unique Spotify identifier assigned to the track
artists	Name(s) of the artist(s) who performed the track
album_name	Name of the album the track belongs to
track_name	Title of the track
popularity	Popularity score of the track (0-100)
duration_ms	Length of the track in milliseconds
explicit	Whether the track contains explicit content (true/false)
danceability	How suitable the track is for dancing (0.0-1.0)
energy	Perceptual measure of intensity and activity (0.0-1.0)
key	Estimated overall musical key of the track

loudness	Overall loudness of the track in decibels (dB)
mode	Modality of the track (major = 1, minor = 0)
speechiness	Presence of spoken words in the track (0.0-1.0)
acousticness	Confidence measure of whether the track is acoustic (0.0-1.0)
instrumentalness	Predicts whether a track contains no vocal content (0.0-1.0)
liveness	Detects the presence of a live audience in the recording (0.0-1.0)
valence	Musical positiveness conveyed by the track (0.0-1.0)
tempo	Estimated tempo of the track in beats per minute (BPM)
time_signature	Estimated overall time signature of the track
track_genre	Genre the track is categorized under

Summary of Data Analysis

Following MapReduce programs and their different design patterns are implemented:

- Filtering and Maximum Value Identification
- Summarization and Aggregation
- Counting and Ranking

Following analysis can be performed on the Spotify tracks dataset using MapReduce:

- Identify the most popular song based on the track popularity value.
- Calculate popularity summarization for different music genres, including minimum popularity, maximum popularity, average popularity, and total number of tracks.
- Identify the top artists based on the number of tracks associated with each artist.

Following Pig analysis is performed on the Spotify tracks dataset:

- Calculate the Top 10 most popular songs.
- Calculate the Top 10 artists based on the number of tracks.
- Calculate genre-wise statistics, such as the number of tracks and popularity-related aggregations.
- Calculate the Top 10 music genres based on the number of tracks.
- Analyze the distribution of explicit and non-explicit tracks in the Spotify dataset.

MapReduce Analysis

1. Top Popular Song

Returns the song with the highest popularity score from the Spotify tracks dataset.

- Driver Class:** sets the Hadoop configuration and configures the Mapper and Reducer classes, and uses a single reducer to guarantee one global maximum.
- Mapper Class:** extracts the track name, artist, and popularity value from each record and emits them as intermediate data.
- Reducer Class:** compares all popularity values received and identifies the track with the maximum popularity value.

Driver Class

```
public class TopPopularSong {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: TopPopularSong <input> <output>");
            System.exit(2);
        }
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Top Popular Song");
        job.setJarByClass(TopPopularSong.class);
        job.setMapperClass(TopPopularSongMapper.class);
        job.setReducerClass(TopPopularSongReducer.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        // One reducer guarantees one global maximum
        job.setNumReduceTasks(1);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Mapper Class

```
public class TopPopularSongMapper extends Mapper<Object, Text, Text, IntWritable> {
    private final Text song = new Text();
    private final IntWritable popularity = new IntWritable();
    @Override
    public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
        String line = value.toString();
        // Skip header
        if (line.startsWith("track_id,")) {
            return;
        }
        List<String> fields = parseCSVLine(line);
        // Spotify dataset has 20 columns
        if (fields.size() < 20) {
            return;
        }
        try {
            // Column 1 = artists, Column 3 = track_name, Column 4 = popularity
            String artists = fields.get(1).trim();
            String trackName = fields.get(3).trim();
            int pop = Integer.parseInt(fields.get(4).trim());
            song.set(trackName + "\t" + artists);
            popularity.set(pop);
            context.write(song, popularity);
        } catch (NumberFormatException e) {
            // Ignore invalid popularity values
        }
    }
}
// CSV parser that handles commas inside quoted fields
private List<String> parseCSVLine(String line) {
    List<String> fields = new ArrayList<>();
    StringBuilder current = new StringBuilder();
    boolean insideQuotes = false;
    for (int i = 0; i < line.length(); i++) {
        char c = line.charAt(i);
        if (c == '"') {
            // Handle escaped double quote ""
            if (insideQuotes && i + 1 < line.length() && line.charAt(i + 1) == '"') {
                current.append('"');
                i++;
            } else {
                insideQuotes = !insideQuotes;
            }
        } else if (c == ',' && !insideQuotes) {
            fields.add(current.toString());
            current.setLength(0);
        } else {
            current.append(c);
        }
    }
    fields.add(current.toString());
    return fields; } }
```

Reducer Class

```
public class TopPopularSongReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private Text topSong = new Text();
    private int maxPopularity = -1;
    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException,
        InterruptedException {
        for (IntWritable value : values) {
            int popularity = value.get();
            if (popularity > maxPopularity) {
                maxPopularity = popularity;
                topSong.set(key);
            }
        }
    }
    @Override
    protected void cleanup(Context context) throws IOException, InterruptedException {
        if (maxPopularity >= 0) {
            context.write(topSong, new IntWritable(maxPopularity));
        }
    }
}
```

Execution

```
samin@samin1212:~/spotify-BDAL-project/mapreduce/MR1_TopPopularSong$ hadoop jar TopPopularSong.jar com.neu.bigdata.TopPopularSong /spotify/input/spotify_tracks.csv /spotify/output/mr1
2026-09-13 21:42:42,020 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2026-09-13 21:42:42,795 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-09-13 21:42:42,956 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/samin/.staging/job_1789308685861_0006
2026-09-13 21:42:43,062 INFO input.FileInputFormat: Total input files to process : 1
2026-09-13 21:42:44,291 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-13 21:42:45,006 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789308685861_0006
2026-09-13 21:42:45,007 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-13 21:42:45,394 INFO conf.Configuration: resource-types.xml not found
2026-09-13 21:42:45,395 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-13 21:42:45,707 INFO impl.YarnClientImpl: Submitted application application_1789308685861_0006
2026-09-13 21:42:45,761 INFO mapreduce.Job: The url to track the job: http://samin1212:8088/proxy/application_1789308685861_0006/
2026-09-13 21:42:45,762 INFO mapreduce.Job: Running job: job_1789308685861_0006
2026-09-13 21:42:58,076 INFO mapreduce.Job: Job job_1789308685861_0006 running in uber mode : false
2026-09-13 21:42:58,079 INFO mapreduce.Job: map 0% reduce 0%
2026-09-13 21:43:09,392 INFO mapreduce.Job: map 100% reduce 0%
2026-09-13 21:43:19,617 INFO mapreduce.Job: map 100% reduce 100%
2026-09-13 21:43:21,701 INFO mapreduce.Job: Job job_1789308685861_0006 completed successfully
2026-09-13 21:43:21,886 INFO mapreduce.Job: Counters: 54
```

Output

```
samin@samin1212:~/spotify-BDAL-project/mapreduce/MR1_TopPopularSong$ hdfs dfs -cat /spotify/output/mr1/part-r-000000
Unholy (feat. Kim Petras)      Sam Smith;Kim Petras      100
```

2. Popularity Summarization

Returns a popularity summary for each music genre in the Spotify tracks dataset, including the minimum popularity, maximum popularity, average popularity, and the total number of tracks per genre.

- i. **Driver Class:** sets the Hadoop configuration and configures the Mapper and Reducer classes.
- ii. **Mapper Class:** extracts the genre and popularity value from each record and emits the genre as the key.
- iii. **Reducer Class:** groups all popularity values belonging to the same genre and calculates the minimum, maximum, average, and count.

Driver Class

```
public class PopularitySummarization {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: PopularitySummarization <input> <output>");
            System.exit(2);
        }
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Spotify Popularity Summarization");
        job.setJarByClass(PopularitySummarization.class);
    }
}
```

```

        job.setMapperClass(PopularitySummaryMapper.class);
        job.setReducerClass(PopularitySummaryReducer.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Mapper Class

```

public class PopularitySummaryMapper extends Mapper<Object, Text, Text, IntWritable> {
    private final Text genre = new Text();
    private final IntWritable popularity = new IntWritable();
    @Override
    public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
        String line = value.toString();
        // Skip header
        if (line.startsWith("track_id,")) {
            return;
        }
        List<String> fields = parseCSVLine(line);
        // Spotify dataset has 20 columns
        if (fields.size() < 20) {
            return;
        }
        try {
            // Column 4 = popularity, Column 19 = track_genre
            int pop = Integer.parseInt(fields.get(4).trim());
            String trackGenre = fields.get(19).trim();
            if (!trackGenre.isEmpty()) {
                genre.set(trackGenre);
                popularity.set(pop);
                context.write(genre, popularity);
            }
        } catch (NumberFormatException e) {
            // Ignore invalid popularity values
        }
    }
    private List<String> parseCSVLine(String line) {
        List<String> fields = new ArrayList<>();
        StringBuilder current = new StringBuilder();
        boolean insideQuotes = false;
        for (int i = 0; i < line.length(); i++) {
            char c = line.charAt(i);
            if (c == '"') {
                if (insideQuotes && i + 1 < line.length() && line.charAt(i + 1) == '"') {
                    current.append('');
                    i++;
                } else {
                    insideQuotes = !insideQuotes;
                }
            } else if (c == ',' && !insideQuotes) {
                fields.add(current.toString());
                current.setLength(0);
            } else {
                current.append(c);
            }
        }
        fields.add(current.toString());
        return fields;
    }
}

```

Reducer Class

```
public class PopularitySummaryReducer extends Reducer<Text, IntWritable, Text, Text> {
    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException,
    InterruptedException {
        int min = Integer.MAX_VALUE;
        int max = Integer.MIN_VALUE;
        long sum = 0;
        long count = 0;
        for (IntWritable value : values) {
            int popularity = value.get();
            if (popularity < min) min = popularity;
            if (popularity > max) max = popularity;
            sum += popularity;
            count++;
        }
        if (count > 0) {
            double average = (double) sum / count;
            String result = min + "\t" + max + "\t" + String.format("%.2f", average) + "\t" + count;
            context.write(key, new Text(result));
        }
    }
}
```

Execution

```
samin@samin1212:~/spotify-BDAL-project/mapreduce/MR2_PopularitySummarization$ hadoop jar PopularitySummarization.jar \
com.neu.bigdata.PopularitySummarization \
/spotify/input/spotify_tracks.csv \
/spotify/output/mr2
2026-09-13 21:59:24,837 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2026-09-13 21:59:26,357 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application
with ToolRunner to remedy this.
2026-09-13 21:59:26,505 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/samin/.staging/job_1789308685861_0007
2026-09-13 21:59:27,707 INFO input.FileInputFormat: Total input files to process : 1
2026-09-13 21:59:28,208 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-13 21:59:29,438 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789308685861_0007
2026-09-13 21:59:29,438 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-13 21:59:30,080 INFO conf.Configuration: resource-types.xml not found
2026-09-13 21:59:30,081 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-13 21:59:31,448 INFO impl.YarnClientImpl: Submitted application application_1789308685861_0007
2026-09-13 21:59:31,517 INFO mapreduce.Job: The url to track the job: http://samin1212:8088/proxy/application_1789308685861_0007/
2026-09-13 21:59:31,517 INFO mapreduce.Job: Running job: job_1789308685861_0007
2026-09-13 22:00:07,296 INFO mapreduce.Job: Job job_1789308685861_0007 running in uber mode : false
2026-09-13 22:00:07,302 INFO mapreduce.Job: map 0% reduce 0%
2026-09-13 22:00:37,193 INFO mapreduce.Job: map 100% reduce 0%
2026-09-13 22:01:10,343 INFO mapreduce.Job: map 100% reduce 100%
2026-09-13 22:01:13,405 INFO mapreduce.Job: Job job_1789308685861_0007 completed successfully
2026-09-13 22:01:13,610 INFO mapreduce.Job: Counters: 54
```

Output

```
samin@samin1212:~/spotify-BDAL-project/mapreduce/MR2_PopularitySummarization$ hdfs dfs -cat /spotify/output/mr2/part-r-00000 | head -20
acoustic      0      82      42.48      1000
afrobeat      0      75      24.40      1000
alt-rock      0      93      33.94      1000
alternative   0      93      24.34      1000
ambient 0     84      44.19      1000
anime 0      83      48.77      1000
black-metal   0      58      22.45      1000
bluegrass     0      69      25.68      1000
blues 0      84      31.19      1000
brazil 0      82      44.67      1000
breakbeat     0      66      20.12      1000
british 0     87      43.80      1000
cantopop      0      72      34.74      1000
chicago-house 0     78      12.34      1000
children      0      77      35.05      1000
chill 0      93      53.65      1000
classical     0      79      13.06      1000
club 0      85      33.34      1000
comedy 19     75      24.63      1000
country 0     89      17.03      1000
samin@samin1212:~/spotify-BDAL-project/mapreduce/MR2_PopularitySummarization$ █
```

3. Top Artists

Returns the artists with the highest number of tracks in the Spotify tracks dataset, ranked by average popularity.

- i. **Driver Class:** sets the Hadoop configuration and configures the Mapper and Reducer classes.
- ii. **Mapper Class:** extracts the artist name(s) and popularity value from each record and emits them as a Text key/value pair.
- iii. **Reducer Class:** aggregates the track count and average popularity per artist and emits only the Top 10 ranked artists.

Driver Class

```
public class TopArtists {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: TopArtists <input> <output>");
            System.exit(2);
        }
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Top 10 Artists by Average Popularity");
        job.setJarByClass(TopArtists.class);
        job.setMapperClass(TopArtistsMapper.class);
        job.setReducerClass(TopArtistsReducer.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        job.setNumReduceTasks(1);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Mapper Class

```
public class TopArtistsMapper extends Mapper<Object, Text, Text, Text> {
    private final Text outputKey = new Text("ALL");
    private final Text outputValue = new Text();
    @Override
    public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
        String line = value.toString();
        if (line.startsWith("track_id,artists,")) {
            return;
        }
        String[] fields = parseCSVLine(line);
        if (fields.length <= 4) {
            return;
        }
        String artistsField = fields[1].trim();
        String popularityText = fields[4].trim();
        if (artistsField.isEmpty()) {
            return;
        }
        try {
            int popularity = Integer.parseInt(popularityText);
            List<String> artists = parseArtists(artistsField);
            for (String artist : artists) {
                artist = artist.trim();
                if (!artist.isEmpty()) {
                    outputValue.set(artist + "\t" + popularity);
                    context.write(outputKey, outputValue);
                }
            }
        } catch (NumberFormatException e) {
            // Ignore invalid popularity values
        }
    }
    private List<String> parseArtists(String text) {
        List<String> artists = new ArrayList<>();
        text = text.trim();
        if (text.startsWith("[") && text.endsWith("]")) {
            text = text.substring(1, text.length() - 1);
        }
        if (text.contains(";")) {
            String[] parts = text.split(";");
            for (String part : parts) {
                part = cleanArtist(part);
                if (!part.isEmpty()) {
                    artists.add(part);
                }
            }
        }
        return artists;
    }
    boolean insideQuotes = false;
    char quoteChar = 0;
}
```

```

        StringBuilder current = new StringBuilder();
        for (int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);
            if (c == '\'' || c == '\"') {
                if (!insideQuotes) {
                    insideQuotes = true;
                    quoteChar = c;
                } else if (c == quoteChar) {
                    insideQuotes = false;
                } else {
                    current.append(c);
                }
            } else if (c == ',' && !insideQuotes) {
                String artist = cleanArtist(current.toString());
                if (!artist.isEmpty()) {
                    artists.add(artist);
                }
                current.setLength(0);
            } else {
                current.append(c);
            }
        }
        String artist = cleanArtist(current.toString());
        if (!artist.isEmpty()) {
            artists.add(artist);
        }
        return artists;
    }

    private String cleanArtist(String artist) {
        artist = artist.trim();
        artist = artist.replaceAll("^['\"]+|['\"]+$", "");
        return artist.trim();
    }

    private String[] parseCSVLine(String line) {
        List<String> fields = new ArrayList<>();
        StringBuilder current = new StringBuilder();
        boolean insideQuotes = false;
        for (int i = 0; i < line.length(); i++) {
            char c = line.charAt(i);
            if (c == '\"') {
                if (insideQuotes && i + 1 < line.length() && line.charAt(i + 1) == '\"') {
                    current.append('\"');
                    i++;
                } else {
                    insideQuotes = !insideQuotes;
                }
            } else if (c == ',' && !insideQuotes) {
                fields.add(current.toString());
                current.setLength(0);
            } else {
                current.append(c);
            }
        }
        fields.add(current.toString());
        return fields.toArray(new String[0]);
    }
}

```

Reducer Class

```

public class TopArtistsReducer extends Reducer<Text, Text, Text, Text> {
    private static class ArtistStats {
        int count;
        long totalPopularity;
        ArtistStats() { count = 0; totalPopularity = 0; }
        void add(int popularity) { count++; totalPopularity += popularity; }
        double average() { return (double) totalPopularity / count; }
    }

    @Override
    public void reduce(Text key, Iterable<Text> values, Context context) throws IOException,
        InterruptedException {
        Map<String, ArtistStats> artists = new HashMap<>();
        for (Text value : values) {
            String[] parts = value.toString().split("\\t", 2);
            if (parts.length != 2) continue;
            String artist = parts[0].trim();

```

```

    try {
        int popularity = Integer.parseInt(parts[1].trim());
        ArtistStats stats = artists.get(artist);
        if (stats == null) {
            stats = new ArtistStats();
            artists.put(artist, stats);
        }
        stats.add(popularity);
    } catch (NumberFormatException e) {
        // Ignore invalid popularity values
    }
}
// Sort artists by average popularity, highest first
List<Map.Entry<String, ArtistStats>> ranking = new ArrayList<>(artists.entrySet());
ranking.sort(
    Comparator.<Map.Entry<String, ArtistStats>>comparingDouble(entry ->
entry.getValue().average())
        .reversed()
        .thenComparing(Map.Entry::getKey)
);
// Output only Top 10
int limit = Math.min(10, ranking.size());
for (int i = 0; i < limit; i++) {
    Map.Entry<String, ArtistStats> entry = ranking.get(i);
    String artist = entry.getKey();
    ArtistStats stats = entry.getValue();
    context.write(new Text(artist), new Text(stats.count + "\t" + String.format("%.2f",
stats.average())));
}
}
}

```

Execution

```

samin@samin1212:~/spotify-BDAL-project/mapreduce/MR3_TopArtists$ hadoop jar TopArtists.jar com.neu.bigdata.TopArtists /spotify/input /spotify/output/mr3
2026-09-13 22:28:22,045 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2026-09-13 22:28:23,081 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application
with ToolRunner to remedy this.
2026-09-13 22:28:23,089 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/samin/.staging/job_1789308685861_0009
2026-09-13 22:28:24,399 INFO input.FileInputFormat: Total input files to process : 1
2026-09-13 22:28:24,758 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-13 22:28:25,269 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789308685861_0009
2026-09-13 22:28:25,269 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-13 22:28:25,661 INFO conf.Configuration: resource-types.xml not found
2026-09-13 22:28:25,661 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-13 22:28:25,752 INFO impl.YarnClientImpl: Submitted application application_1789308685861_0009
2026-09-13 22:28:25,796 INFO mapreduce.Job: The url to track the job: http://samin1212:8088/proxy/application_1789308685861_0009/
2026-09-13 22:28:25,798 INFO mapreduce.Job: Running job: job_1789308685861_0009
2026-09-13 22:28:52,417 INFO mapreduce.Job: Job job_1789308685861_0009 running in uber mode : false
2026-09-13 22:28:52,419 INFO mapreduce.Job: map 0% reduce 0%
2026-09-13 22:29:03,687 INFO mapreduce.Job: map 100% reduce 0%
2026-09-13 22:29:13,851 INFO mapreduce.Job: map 100% reduce 100%
2026-09-13 22:29:22,006 INFO mapreduce.Job: Job job_1789308685861_0009 completed successfully
2026-09-13 22:29:22,451 INFO mapreduce.Job: Counters: 54
File Counter Counters

```

Output

```

samin@samin1212:~/spotify-BDAL-project/mapreduce/MR3_TopArtists$ hdfs dfs -cat /spotify/output/mr3/part-r-00000
Bizarrap      1      99.00
Quevedo 1      99.00
Harry Styles  3      92.00
Luar La L     4      90.50
Beach Weather 1      90.00
Pablo Pesadilla 1      88.00
Polimá Westcoast 1      88.00
Olivia Rodrigo 5      87.40
BYOR 4      87.00
Lewis Capaldi 1      87.00
samin@samin1212:~/spotify-BDAL-project/mapreduce/MR3_TopArtists$ █

```

Pig Analysis

1. Top 10 Popular Songs

This Pig analysis identifies the Top 10 most popular songs from the Spotify tracks dataset. The dataset is loaded from HDFS, and the required track information and popularity values are extracted. The tracks are sorted in descending order based on their popularity score, and the top 10 records are selected and stored in the output directory.

Script

```
songs = LOAD '/spotify/input/spotify_tracks.csv'
USING org.apache.pig.piggybank.storage.CSVExcelStorage(',')
AS (
  track_id:chararray,
  artists:chararray,
  album_name:chararray,
  track_name:chararray,
  popularity:int,
  duration_ms:int,
  explicit:boolean,
  danceability:double,
  energy:double,
  key:int,
  loudness:double,
  mode:int,
  speechiness:double,
  acousticness:double,
  instrumentalness:double,
  liveness:double,
  valence:double,
  tempo:double,
  time_signature:int,
  track_genre:chararray
);
songs_no_header = FILTER songs BY track_id != 'track_id';
grouped_songs = GROUP songs_no_header BY track_id;
unique_songs = FOREACH grouped_songs GENERATE
  group AS track_id,
  MAX(songs_no_header.track_name) AS track_name,
  MAX(songs_no_header.artists) AS artists,
  MAX(songs_no_header.popularity) AS popularity,
  MAX(songs_no_header.track_genre) AS track_genre;
sorted_songs = ORDER unique_songs BY popularity DESC, track_name ASC;
top10 = LIMIT sorted_songs 10;
result = FOREACH top10 GENERATE track_name, artists, popularity, track_genre;
STORE result INTO '/spotify/pig/output/top10_songs' USING PigStorage('\t');
```

Output

```
samin@samin1212:~/spotify-BDAL-project/pig/Pig1_TopSongs$ printf "%-5s %-45s %-35s %-10s %-20s\n" "Rank" "Track Name" "Artist" "Popularity" "Genre"
printf "%-5s %-45s %-35s %-10s %-20s\n" "----" "-----" "-----" "-----" "-----"

hdfs dfs -cat /spotify/pig/output/top10_songs/part-r-00000 | \
awk -F '\t' '{printf "%-5d %-45s %-35s %-10s %-20s\n", NR, $1, $2, $3, $4}'
Rank  Track Name                                     Artist                                     Popularity  Genre
----  -
1     Unholy (feat. Kim Petras)                         Sam Smith;Kim Petras                     100         pop
2     Quevedo: Bzrp Music Sessions, Vol. 52              Bizarrap;Quevedo                         99          hip-hop
3     I'm Good (Blue)                                    David Guetta;Bebe Rexha                  98          pop
4     La Bachata                                         Manuel Turizo                             98          reggaeton
5     Me Porto Bonito                                    Bad Bunny;Chencho Corleone               97          reggaeton
6     Titi Me Preguntó                                    Bad Bunny                                 97          reggaeton
7     Efecto                                              Bad Bunny                                 96          reggaeton
8     I Ain't Worried                                    OneRepublic                              96          rock
9     Under The Influence                               Chris Brown                              96          pop
10    As It Was                                           Harry Styles                             95          pop
```

2. Top 10 Artists

This Pig analysis identifies the Top 10 artists based on the number of tracks available in the Spotify dataset. The dataset is grouped by artist name, and the number of tracks associated with each artist is counted. The artists are then sorted in descending order according to their average popularity, and the Top 10 artists are selected.

Script

```
songs = LOAD '/spotify/input/spotify_tracks.csv'
USING org.apache.pig.piggybank.storage.CSVExcelStorage(',')
AS (
  track_id:chararray,
  artists:chararray,
  album_name:chararray,
  track_name:chararray,
  popularity:int,
  duration_ms:int,
  explicit:boolean,
```

```

    danceability:double,
    energy:double,
    key:int,
    loudness:double,
    mode:int,
    speechiness:double,
    acousticness:double,
    instrumentalness:double,
    liveness:double,
    valence:double,
    tempo:double,
    time_signature:int,
    track_genre:chararray
);
songs_no_header = FILTER songs BY track_id != 'track_id';
artist_tracks = FOREACH songs_no_header GENERATE
    FLATTEN(TOKENIZE(artists, ';')) AS artist, popularity;
artist_tracks = FILTER artist_tracks BY artist IS NOT NULL AND TRIM(artist) != '';
grouped_artists = GROUP artist_tracks BY TRIM(artist);
artist_stats = FOREACH grouped_artists GENERATE
    group AS artist,
    COUNT(artist_tracks) AS number_of_tracks,
    AVG(artist_tracks.popularity) AS average_popularity;
sorted_artists = ORDER artist_stats BY average_popularity DESC, artist ASC;
top10 = LIMIT sorted_artists 10;
STORE top10 INTO '/spotify/pig/output/top10_artists' USING PigStorage('\t');

```

Output

```

samin@samin1212:~/spotify-BDAL-project/pig/Pig2_TopArtists$ printf "%-5s %-35s %-10s %-20s\n" "Rank" "Artist" "Tracks" "Average Popularity"
printf "%-5s %-35s %-10s %-20s\n" "-----" "-----" "-----" "-----"
hdfs dfs -cat /spotify/pig/output/top10_artists/part-r-00000 | \
awk -F '\t' '{printf "%-5d %-35s %-10s %-20.2f\n", NR, $1, $2, $3}'
Rank  Artist                               Tracks  Average Popularity
-----
1     Bizarrap                               1       99.00
2     Quevedo                                1       99.00
3     Harry Styles                           3       92.00
4     Luar La L                              4       90.50
5     Beach Weather                          1       90.00
6     Pablo Pesadilla                        1       88.00
7     Polimá Westcoast                       1       88.00
8     Olivia Rodrigo                         5       87.40
9     BYOR                                    4       87.00
10    Lewis Capaldi                          1       87.00
samin@samin1212:~/spotify-BDAL-project/pig/Pig2_TopArtists$ █

```

3. Genre Statistics

This Pig analysis performs genre-wise statistical analysis on the Spotify tracks dataset. The tracks are grouped according to their music genre, and statistical measures (track count, average, maximum, and minimum popularity) are calculated for each genre.

Script

```

songs = LOAD '/spotify/input/spotify_tracks.csv'
USING org.apache.pig.piggybank.storage.CSVExcelStorage(',')
AS (
    track_id:chararray,
    artists:chararray,
    album_name:chararray,
    track_name:chararray,
    popularity:int,
    duration_ms:int,
    explicit:boolean,
    danceability:double,
    energy:double,
    key:int,
    loudness:double,
    mode:int,
    speechiness:double,
    acousticness:double,
    instrumentalness:double,
    liveness:double,
    valence:double,
    tempo:double,
    time_signature:int,

```

```

    track_genre:chararray
);
songs_no_header = FILTER songs BY track_id != 'track_id';
genre_group = GROUP songs_no_header BY track_genre;
genre_stats = FOREACH genre_group GENERATE
    group AS genre,
    COUNT(songs_no_header) AS number_of_tracks,
    AVG(songs_no_header.popularity) AS average_popularity,
    MAX(songs_no_header.popularity) AS maximum_popularity,
    MIN(songs_no_header.popularity) AS minimum_popularity;
sorted_genres = ORDER genre_stats BY genre ASC;
STORE sorted_genres INTO '/spotify/pig/output/genre_statistics' USING PigStorage('\t');

```

Output

```

samin@samin1212:~/spotify-BDAL-project/pig/Pig3_GenreStatistics$ printf "%-20s %-10s %-20s %-10s %-10s\n" "Genre" "Tracks" "Average" "Maximum" "Minimum"
printf "%-20s %-10s %-20s %-10s %-10s\n" "-----" "-----" "-----" "-----" "-----"
hdfs dfs -cat /spotify/pig/output/genre_statistics/part-r-00000 | \
awk -F '\t' '{printf "%-20s %-10s %-20.2f %-10s %-10s\n", $1, $2, $3, $4, $5}'
Genre          Tracks    Average      Maximum    Minimum
-----
acoustic        1000      42.48        82         0
afrobeat        1000      24.40        75         0
alt-rock        1000      33.94        93         0
alternative     1000      24.34        93         0
ambient         1000      44.19        84         0
anime           1000      48.77        83         0
black-metal     1000      22.45        58         0
bluegrass       1000      25.68        69         0
blues           1000      31.19        84         0
brazil          1000      44.67        82         0
breakbeat       1000      26.12        66         0
british         1000      43.80        87         0
cantopop        1000      34.74        72         0
chicago-house  1000      12.34        78         0
children        1000      35.05        77         0
chill           1000      53.65        93         0
classical       1000      13.05        79         0
club            1000      33.34        85         0
comedy          1000      24.63        75         19
country         1000      17.03        89         0

```

4. Top 10 Music Genres

This Pig analysis identifies the Top 10 music genres based on average popularity in the Spotify dataset, using the output of the genre statistics analysis above. The genres are sorted in descending order by average popularity, and the Top 10 genres are selected.

Script

```

genre_stats = LOAD '/spotify/pig/output/genre_statistics'
USING PigStorage('\t')
AS (
    genre:chararray,
    track_count:int,
    average_popularity:double,
    max_popularity:int,
    min_popularity:int
);
sorted_genres = ORDER genre_stats BY average_popularity DESC;
top10_genres = LIMIT sorted_genres 10;
STORE top10_genres INTO '/spotify/pig/output/top10_genres' USING PigStorage('\t');

```

Output

```

samin@samin1212:~/spotify-BDAL-project/pig/Pig4_TopGenres$ hdfs dfs -cat /spotify/pig/output/top10_genres/*
pop-film        1000    59.283    80     0
k-pop           1000    56.896    88     0
chill           1000    53.651    93     0
sad             1000    52.379    83     0
grunge          1000    49.594    85     0
indian          1000    49.539    88     0
anime           1000    48.772    83     0
emo             1000    48.128    87     0
sertanejo       1000    47.866    63     0
pop             1000    47.576    100    0

```

5. Explicit Content Analysis

This Pig analysis examines the distribution of explicit and non-explicit tracks in the Spotify tracks dataset. The records are grouped according to their explicit-content status, and the number of tracks and popularity statistics are calculated for each category.

Script

```
songs = LOAD '/spotify/input/spotify_tracks.csv'
USING org.apache.pig.piggybank.storage.CSVExcelStorage(',')
AS (
  track_id:chararray,
  artists:chararray,
  album_name:chararray,
  track_name:chararray,
  popularity:int,
  duration_ms:int,
  explicit:boolean,
  danceability:double,
  energy:double,
  key:int,
  loudness:double,
  mode:int,
  speechiness:double,
  acousticness:double,
  instrumentalness:double,
  liveness:double,
  valence:double,
  tempo:double,
  time_signature:int,
  track_genre:chararray
);
songs_no_header = FILTER songs BY track_id != 'track_id';
grouped_explicit = GROUP songs_no_header BY explicit;
explicit_stats = FOREACH grouped_explicit GENERATE
  group AS explicit,
  COUNT(songs_no_header) AS number_of_tracks,
  AVG(songs_no_header.popularity) AS average_popularity,
  MAX(songs_no_header.popularity) AS max_popularity,
  MIN(songs_no_header.popularity) AS min_popularity;
sorted_explicit = ORDER explicit_stats BY explicit ASC;
STORE sorted_explicit INTO '/spotify/pig/output/explicit_analysis' USING PigStorage('\t');
```

Output

```
samin@samin1212:~/spotify-BDAL-project/pig/Pig5_ExplicitAnalysis$ {
printf "Explicit\tNumber_of_Tracks\tAverage_Popularity\tMax_Popularity\tMin_Popularity\n"
hdfs dfs -cat /spotify/pig/output/explicit_analysis/* | sort -r | awk -F'\t' 'BEGIN{OFS="\t"} {printf "%s\t%s\t%.2f\t%s\t%s\n", $1, $2, $3, $4, $5}'
} | column -t -s $'\t'
Explicit Number_of_Tracks Average_Popularity Max_Popularity Min_Popularity
true 9747 36.45 98 0
false 104253 32.94 100 0
samin@samin1212:~/spotify-BDAL-project/pig/Pig5_ExplicitAnalysis$
```

MapReduce vs Pig Comparison

The Spotify tracks dataset was analyzed using both Apache Hadoop MapReduce and Apache Pig, to verify whether both technologies produced consistent results for similar analytical tasks and to compare their approaches to Big Data processing.

1. Top Song Analysis

Comparing MapReduce MR1 (Top Popular Song) with Pig1 (Top 10 Popular Songs): both analyses identified the same most popular song, "Unholy (feat. Kim Petras)", with a popularity score of 100. MapReduce required Java Mapper, Reducer, and Driver classes, while Pig performed the same analysis with a shorter script involving loading, sorting, and limiting the dataset. Both approaches produced the same result.

2. Top Artists Analysis

Comparing MapReduce MR3 (Top Artists) with Pig2 (Top 10 Artists): both analyses produced the same Top 10 artist ranking based on track count and average popularity, with Bizarrap ranked first (1 track, average popularity 99.00). MapReduce and Pig produced identical rankings, showing that both technologies can effectively perform grouping, counting, sorting, and ranking on the dataset.

3. Genre Statistics Analysis

Comparing MapReduce MR2 (Popularity Summarization) with Pig3 (Genre Statistics): both calculated the minimum, maximum, average popularity, and total track count for each genre. For example, for the "acoustic" genre, MR2 reported a minimum of 0, maximum of 82, average of 42.48, and 1000 total tracks, while Pig3 reported the same 1000 tracks, average popularity of 42.483, maximum of 82, and minimum of 0. The results are equivalent, differing only in column order and decimal precision.

4. Additional Pig Analyses

Apache Pig was also used to perform two analyses that were not directly implemented using MapReduce:

- Pig4 (Top 10 Genres): identifies the Top 10 music genres by average popularity, showing which genres have the highest overall popularity among their tracks.
- Pig5 (Explicit Content Analysis): compares explicit and non-explicit tracks using number of tracks, average popularity, maximum popularity, and minimum popularity.

Overall Comparison

The analyses show that Apache Hadoop MapReduce and Apache Pig produced consistent and equivalent results for the tasks implemented using both technologies. MapReduce gives greater control over the processing logic through separate Driver, Mapper, and Reducer classes, but requires more programming effort and a larger amount of code. Apache Pig provides a higher-level approach, letting filtering, grouping, counting, sorting, aggregation, and ranking be implemented with shorter and simpler scripts. For the Spotify dataset, both technologies successfully identified the most popular song, analyzed top artists, and calculated genre-based popularity statistics, while Pig additionally provided concise implementations for genre ranking and explicit-content analysis.

Conclusion: MapReduce is suitable when detailed control over distributed processing is required, while Pig is more convenient for high-level data analysis with less code. The consistent results obtained from both approaches validate the correctness and reliability of the implemented Big Data analyses.